哈爾濱工程大學

第 12 讲 / 全 17 讲

C 语言指针用法详解

地址 · 指针变量 · 指针与函数 · 指针与数组及字符串

地址
每个变量都有房间号

指针变量
专门存放地址的变量

指针与函数
让函数真正改到实参

指针与数组
数组名就是首地址

哈尔滨工程大学 · C 语言程序设计　主讲教师：郎大鹏

本节目标

学完这节你能做什么

哈尔滨工程大学

1. 说出地址是什么，明白变量住在内存的哪一间房
2. 会用 & 取地址，用 %p 把地址打印出来看
3. 会定义指针变量，理解星号在定义与使用时含义不同
4. 会用指针作函数参数，让函数改到真正的实参
5. 会用指针遍历数组，知道 p + 1 跳过的是一整个元素
6. 会用指针走完字符串，手写循环求长度、统计字符

01

地址与指针

先搞清地址是什么，再学怎么把地址存进变量里。

内存房间图

地址 = 房间号

取地址与解引用

两个符号互逆

哈爾濱工程大学

变量住在内存的房间里

房间号就是地址

哈尔滨工程大学

程序一运行，每个变量都被安排进一间带编号的内存小房间：房间里住的是值，房间号就是地址。变量名只是我们贴在门上的便利贴。

地址 2000	地址 2004	地址 2008	地址 2012	地址 2016
10	**20**	**3**	**8**	**0**
变量名 a	变量名 b	变量名 x	变量名 y	变量名 sum
房间里住的值	房间里住的值	房间里住的值	房间里住的值	房间里住的值

int 占 4 个字节，所以相邻两间房的编号差 4；记住房间号，就能绕开变量名直接找到那一间。

用 & 拿到房间号

新格式符 %p

哈尔滨工程大学

```
int a = 10;
double d = 3.5;
// & 读作 " 取地址 "
printf("%p", &a);
// 屏幕上看到 0x7ffd...2000
printf("%p", &d);
// 另一间房 0x7ffd...2008
printf("%d", a);
// 直接问房间里住着什么
scanf("%d", &a);
// 早就用过 & 了!
printf("%d", *(&a));  // 还是 10
```

取地址运算符 &

写在变量名前面，得到这个变量所在房间的编号。

写在谁前面，就拿谁的房间号。

%p 打印地址

地址通常以十六进制显示，形如 0x7ffd... 。每次运行编号可能不同，不用背。

其实早就见过

scanf 里的 &a 就是把房间号交给 scanf ，让它把读到的数放进那一间。

把地址存进变量里

指针变量的定义

哈爾濱工程大學

```
int a = 10;
int *p;
// 星号说明 p 装的是地址
p = &a;
// 把 a 的房间号交给 p
int *q = &a;
// 定义时直接给初值
int* r = &a;
// 星号靠左靠右都一样
printf("%p", p);
// p 里存的是房间号
```

定义格式

类型后面加一个星号： int *p; 读作 "p 是一个指针变量，专门存放 int 型变量的地址 " 。

星号靠哪边都行

int* p 、 int *p 、 int * p 三种写法完全等价，本课件统一写成 int *p 。

指针自己也是变量

p 自己也有房间、也有地址，屋里住的纸条写着 a 的房间号。可以先定义后赋值。

顺着地址进门：& 与 * 互逆 本节核心

哈尔滨工程大学

拿到房间号还不够，还得拿着它进房间：在指针前面加星号，就表示 " 顺着地址去那一间 " 。

变量 a（房间 2000）

10

房间里住的值

&a 拿房间号 → 2000

***p 拿号进门 → 10**

指针变量 p

2000

纸条上记的门牌号

```
int a = 10;
int *p = &a;
printf("%d", *p);
// 10 顺着地址读 a
*p = 20;
// 顺着地址改 a
```

两个符号的分工

取地址是 " 从名字拿房间号 " ，星号是 " 拿房间号进房间 " ，方向正好相反。

*(&a) 就是 a 本人

先取号再进门，两步互相抵消，写在一起等于什么都没干。

用指针的三条铁律

背下来就少踩坑

哈尔滨工程大学

指针不难，难的是分不清 " 地址 " 和 " 房间里的值 " 。记住下面三条，写指针就不会手抖。

铁律一

指针只装地址

p 里放的是房间号，不是值。

铁律二

先指向再用

先让 p 等于某个地址，再写 *p 。

铁律三

读写都加 *

读值写 *p ，改值也写 *p = 20 。

```
int a = 10;
int *p;
p = &a;
// 铁律二：先让它指向 a 的房间
printf("%d", *p);
// 铁律三：加星号才读到 10
```

指针的类型必须匹配

房间大小要一致

哈尔滨工程大学

```
int a = 10;
double d = 3.5;
int *pi = &a;
// 对：int 地址给 int 指针
double *pd = &d;
// 对：double 地址给 double 指针
int *px = &d;
// 错：类型不匹配，编译器警告
printf("%d", *pi);
// 10
```

为什么必须匹配

进门取东西要先知道房间多大：int 型房间 4 字节，double 型 8 字节，认错了房间号就取错值。

类型还决定步长

后面会看到 p + 1 走多远，完全由指针的类型决定，这也是类型不能乱写的原因。

看到警告别硬跑

编译器的类型警告是善意提醒，改到类型一致再运行，比后面调试省事得多。

02

指针与函数

把地址交给函数，让它改到真正的实参。

值传递的遗憾

swap 为什么没成功

输出型参数

一个函数带回多个结果

哈爾濱工程大学

值传递的遗憾

呼应函数课那个失败的 swap

哈尔滨工程大学

函数课写过一个 swap ，形参里换得天翻地覆，回到 main 打印却纹丝不动 —— 因为传过去的只是复印件。

```
void swap(int x, int y)
{
    int t = x;
    x = y;
    y = t;
}
```

传的是复印件

调用时把 a 、 b 的值抄一份给 x 、 y 。

改的是另一间房

x 、 y 是 swap 自己的局部变量。

当时的运行结果

swap 内部： x = 8 ， y = 3 ，确实换过来了

回到 main 打印： a = 3 ， b = 8 ，白忙一场（ main 里调的是 swap(a, b); ）

传地址：swap 终于成了

本节核心

哈尔滨工程大学

```
void swap(int *x, int *y)
{
    int t;
    t = *x;
    // 进门取出 a 的值
    *x = *y;
    // 进门把 b 写进 a
    *y = t;
}
int a = 3, b = 8;
swap(&a, &b);
// 打印 8 3，真的换了
```

两张纸条指向同一间房

调用 swap(&a, &b) 时，x 记住 a 的房间号，y 记住 b 的房间号。

swap 里写 *x、*y，动手脚的对象就是 main 里那一间。

运行结果：交换成功

输入 3 8 → 输出 8 3

跟上一页同一道题，只改了三个地方。

只改了三处

形参加星号、调用加取地址、函数内加星号。

指针参数的三步套路

照着写就不会错

哈尔滨工程大学

想让函数改到实参，就按这三步写：调用处取地址，形参收地址，函数内加星号进门。

① 取地址

调用处写 swap(&a, &b);

② 收地址

形参写 void swap(int *x)

③ 进门操作

函数内写 *x = *y;

```
swap(&a, &b);                  //  第一步 取地址
void swap(int *x, int *y)     //  第二步 收地址
{
    int t = *x;               //  第三步 进门
    *x = *y;   *y = t;
}
printf("%d %d", a, b);    // 8 3
```

输出型参数：带回多个结果

突破 return 只能带一个

哈尔滨工程大学

```
void maxMin(int a, int b, int *pMax, int *pMin)
{
    if (a > b) {
        *pMax = a;  *pMin = b;
    } else {
        *pMax = b;  *pMin = a;
    }
}
int mx, mn;
maxMin(3, 8, &mx, &mn);
// mx 得到 8，mn 得到 3
```

return 只有一张嘴

一个函数最多只能 return 一个值，想带回两个数就抓瞎。

输出型参数

把结果的地址交进去，函数直接写进调用者的房间。

调用处准备空盒子

mx、mn 不用先赋值，函数会替你填好。

什么时候用指针参数

三种典型场景

哈尔滨工程大学

不是所有参数都要加星号。只在下面三种情况才需要把地址交给函数。

场景一

要改实参

想在函数里改掉调用者的变量，比如交换、清零、翻倍。

```
swap(&a, &b);
```

场景二

带回多个结果

一个大值一个最小值， return 装不下，用输出型参数。

```
maxMin(3, 8, &mx, &mn);
```

场景三

传数组

数组课传的就是首地址，下一部分详细讲。

```
sum(score, 5);
```

反过来：只是借来算一算，就别传地址

求平方、判断素数这类 " 给我一个数、还你一个数 " 的函数，用普通值传递就够了。

```
int square(int x);  // 不需要指针
```

易错对比：值传递与地址传递

四组对照看清差别

哈尔滨工程大学

值传递 · 形参不带星号

```
void swap(int x, int y)
```

收到复印件，改不到 main 里去

地址传递 · 形参带星号

```
void swap(int *x, int *y)
```

收到房间号，能改到真正的实参

调用时忘了取地址

```
swap(a, b);
```

类型对不上，编译器直接报错

调用时带上房间号

```
swap(&a, &b);
```

把两个地址一起递过去

函数里忘了加星号

```
x = y;
```

改的是纸条本身，房间原封不动

函数里进门再动手

```
*x = *y;
```

顺着地址改到房间里的内容

打印时忘了加星号

```
printf("%d", p);
```

输出一串地址，不是想要的值

打印房间里的值

```
printf("%d", *p);
```

加星号进门，看到的才是 10

03

指针与数组及字符串

数组名本来就是地址，指针正好当游标。

数组名即地址

p = a 不加取地址

指针算术与遍历

走到底靠结束符

哈爾濱工程大學

数组名就是首地址

呼应数组课 " 传首地址 "

数组课把数组传给函数时，传的其实是首元素的地址 —— 因为数组名本身就代表这块房间的起点。

a[0] · 地址 2000	a[1] · 地址 2004	a[2] · 地址 2008	a[3] · 地址 2012	a[4] · 地址 2016
3	**1**	**4**	**1**	**5**

```
int a[5] = {3, 1, 4, 1, 5};
int *p;
p = a;              //  数组名就是地址
printf("%p", a);    // 2000
printf("%p", &a[0]);  // 2000  一样
printf("%d", *p);   // 3  第一个元素
```

不用加取地址

写 p = a 就行；写 p = &a 反而是另一种意思。

数组名不能改

a 是这块房间的名字，不能赋值； p 才是可以挪动的游标。

指针算术：p + 1 走多远

跳过一个元素

```
int a[5] = {3, 1, 4, 1, 5};
int *p = a;
printf("%d", *p);
// 3  当前指着  a[0]
printf("%d", *(p + 1));
// 1  往后挪一个元素
p++;
// p  自己挪到  a[1]，地址加了  4
printf("%d", *p);
// 1
printf("%d", *(p + 2));
// 4  也就是  a[3]
```

不是加 1 个字节

p + 1 跳过的是一整个元素，int 型就跳 4 个字节。

步长由类型决定

double 型指针加 1，地址加 8；char 型加 1，地址加 1。

和下标是一回事

a[i] 就是 *(a + i)，两种写法完全等价。

用指针遍历数组

本节核心

把指针当游标：从首地址出发，每轮 p++ 往前挪一格，一路走到末尾。

p 起点	p + 1	p + 2	p + 3	p + 4
3	1	4	1	5

```
int a[5] = {3, 1, 4, 1, 5};
int *p;
int sum = 0;
for (p = a; p < a + 5; p++) {
    sum = sum + *p;
}
printf("%d", sum);    // 14
```

指针法的三个看点

起点： p = a ，指针从首元素出发。

终点： p < a + 5 ，走到第 5 个元素后面就停。

取值： *p 进门拿当前这格的值。

和 for (i = 0; i < 5; i++) 完全等价，只是游标换成了指针。

数组参数的两种写法

完全等价

```
void printArr(int *a, int n)
// 写法一：指针形式
{
    int i;
    for (i = 0; i < n; i++) {
        printf("%d", a[i]);
    }
}
void printArr(int a[], int n)
// 写法二：数组形式，等价
// 调用：printArr(score, 5);
```

编译器眼里一样

形参写成 int a[] 还是 int *a ，收到的都是首地址。

长度要另传

函数里数不出数组有多长，必须再传一个 n 。

改得到原数组

传的是地址，函数里改 a[i] ， main 里的数组跟着变。

字符串的两种写法

字符串可以装在字符数组里，也可以让指针指向它 —— 两种写法的身份并不一样。

数组写法：自己的房间

```
char s[] = "Hello";
```

六个格子由 s 自己保管，内容随便改。

```
s[0] = 'h';   // 合法
```

指针写法：指向常量

```
char *p = "Hello";
```

p 只记住了那串常量的地址，房间不是自己的。

```
p[0] = 'h';   // 危险
```

```
char s[] = "Hello";
char *p = "Hello";
printf("%s", s);   // Hello
printf("%s", p);   // Hello
```

走到底自己会停

碰到结束符就停

数组遍历靠长度，字符串遍历靠最后的结束符 '\0' —— 指针撞上它就停，不用事先数有几个字符。

p = s	p + 1	p + 2	p + 3	p + 4	p + 5
H	e	l	l	o	\0

```
char s[] = "Hello";
char *p;
for (p = s; *p != '\0'; p++) {
    printf("%c", *p);
}
// 输出 Hello，指针自己会停
```

跟数组遍历的三点不同

① 不用知道长度 n ，字符串自己有终点。

② 循环变量是指针 p ，而不是整数 i 。

③ 判断的是当前这格的字符内容。

数组课用 s[i] 数过长度，现在换成 p 走，本质一样。

例题精讲 · 例 1

指针版两数交换

哈尔滨工程大学

例 1 输入两个整数，用指针参数把它们真正交换后输出

```c
#include <stdio.h>
void swap(int *x, int *y) {
    int t = *x;   // 进门先取出
    *x = *y;  *y = t;
}
int main(void) {
    int a, b;
    scanf("%d%d", &a, &b);
    swap(&a, &b);
    printf("%d %d", a, b);
    return 0;
}
```

运行结果

输入： 3 8

输出： 8 3

关键点

形参带星号，调用处取地址，函数内进门读写 —— 三步一步都不能少。

swap 不需要返回值，改的是 main 里那两间房。

scanf 里的 &a 和这里的 &a 是同一个道理。

例题精讲 · 例 2

输出型参数带回两个结果

哈尔滨工程大学

例 2 找数组中的最大值，同时把它的下标也带回来

```
void findMax(int *a, int n, int *pMax, int *pPos) {
    int i;
    *pMax = *a;  *pPos = 0;
    for (i = 1; i < n; i++) {
        if (*(a + i) > *pMax) {
            *pMax = *(a + i);  *pPos = i;
        }
    }
}
int score[5] = {72, 95, 88, 61, 90};
int mx, pos;
findMax(score, 5, &mx, &pos);
```

运行结果

```
score = 72 95 88 61 90
mx = 95 ,  pos = 1
```

关键点

两个输出型参数一起带回结果，return 不用出场。

*(a + i) 与 a[i] 完全等价，这里是 " 指针写法 " 。

mx、pos 在 main 里定义，把地址交给函数填写。

例题精讲 · 例 3

指针遍历字符串

哈尔滨工程大学

例 3 统计字符串中大写字母的个数，用指针一路走到底

```
int countUpper(char *s) {
    int cnt = 0;
    char *p;
    for (p = s; *p != '\0'; p++) {
        if (*p >= 'A') {
            if (*p <= 'Z') {
                cnt++;
            }
        }
    }
    return cnt;
}
```

运行结果

`s = "HeLLo C Lang"`

输出： 4

关键点

循环条件判断的是 *p 这个字符，不是位置。

判断区间用两层 if 嵌套，比写在一起更好读。

形参写成 char *s ，调用时直接传数组名。

例题精讲 · 例 4

手写求长度，不用库函数

哈尔滨工程大学

例 4 自己写函数求字符串长度，一路走到结束符为止

```
int myLen(char *s) {
    int n = 0;
    char *p;
    for (p = s; *p != '\0'; p++) {
        n++;
    }
    return n;
}
char s[] = "Hello";
printf("%d", myLen(s));
// 5，走 6 次撞上结束符就停
```

运行结果

s = "Hello"

输出： 5

走法对照

下标法： for (i = 0; s[i] != '\0'; i++)

指针法： for (p = s; *p != '\0'; p++)

游标从 s + i 变成 p ，其余一点没变。

例题精讲 · 例 5

指针方式累加求和

哈尔滨工程大学

例 5 用指针遍历数组，求全部元素之和

```
int sum(int *a, int n) {
    int *p;
    int s = 0;
    for (p = a; p < a + n; p++) {
        s = s + *p;
    }
    return s;
}
int a[5] = {3, 1, 4, 1, 5};
printf("%d", sum(a, 5));
// 14
```

运行结果

```
a = 3 1 4 1 5
```

输出： 14

关键点

终点写 a + n ，正好停在第 n 个元素后面。

循环里只写 *p ，不用再写 a[p - a] 这种绕弯式子。

形参 int *a 换成 int a[] 效果完全一样。

常见错误盘点

这六个坑最多人踩

哈尔滨工程大学

- ✗ 没指向就解引用：纸条上还没写房间号就进门 `int *p;  *p = 10;`
- ✗ 类型不匹配： double 的地址塞给 int 型指针 `int *p = &d;`
- ✗ 星号漏写：想看值却打印出地址 `printf("%d", p);`
- ✗ 野指针：函数结束后还拿着局部变量的房间号 `return &t;`
- ✗ 走到界外：指针越过数组最后一格还在取值 `*(a + n)`
- ✗ 把数组名当变量：数组名是常量，不能赋值或自增 `a++;  a = p;`

小结

地址是房间号，指针是记房间号的纸条。

取地址进门改值，函数终于能改到实参。

数组名就是首地址，指针当游标一路走到底。

两个符号

取地址与解引用

三步套路

取地址收地址进门

遍历法

数组看长度，字符串看结束符